X.1 Component Architecture

Figure X.1 is the discrete-event simulation framework for AQNet, with its four layers: Configuration & Setup, Application Layer, Core and Infrastructure Services.

The Process/User model holds the AQNet logical processes: `ArrivalSource` SRC; one `Server` class instantiated as N1 (M/M/1/K), L1 (M/M/∞), N2 (M/M/c/K), N3 (M/M/n/n) and N4 (M/M/c); `Sink` SNK; and the `Router`, with $p_{\mathrm{N2}\to\mathrm{N3}} = 0.6$ and $p_{\mathrm{N2}\to\mathrm{N4}} = 0.4$.

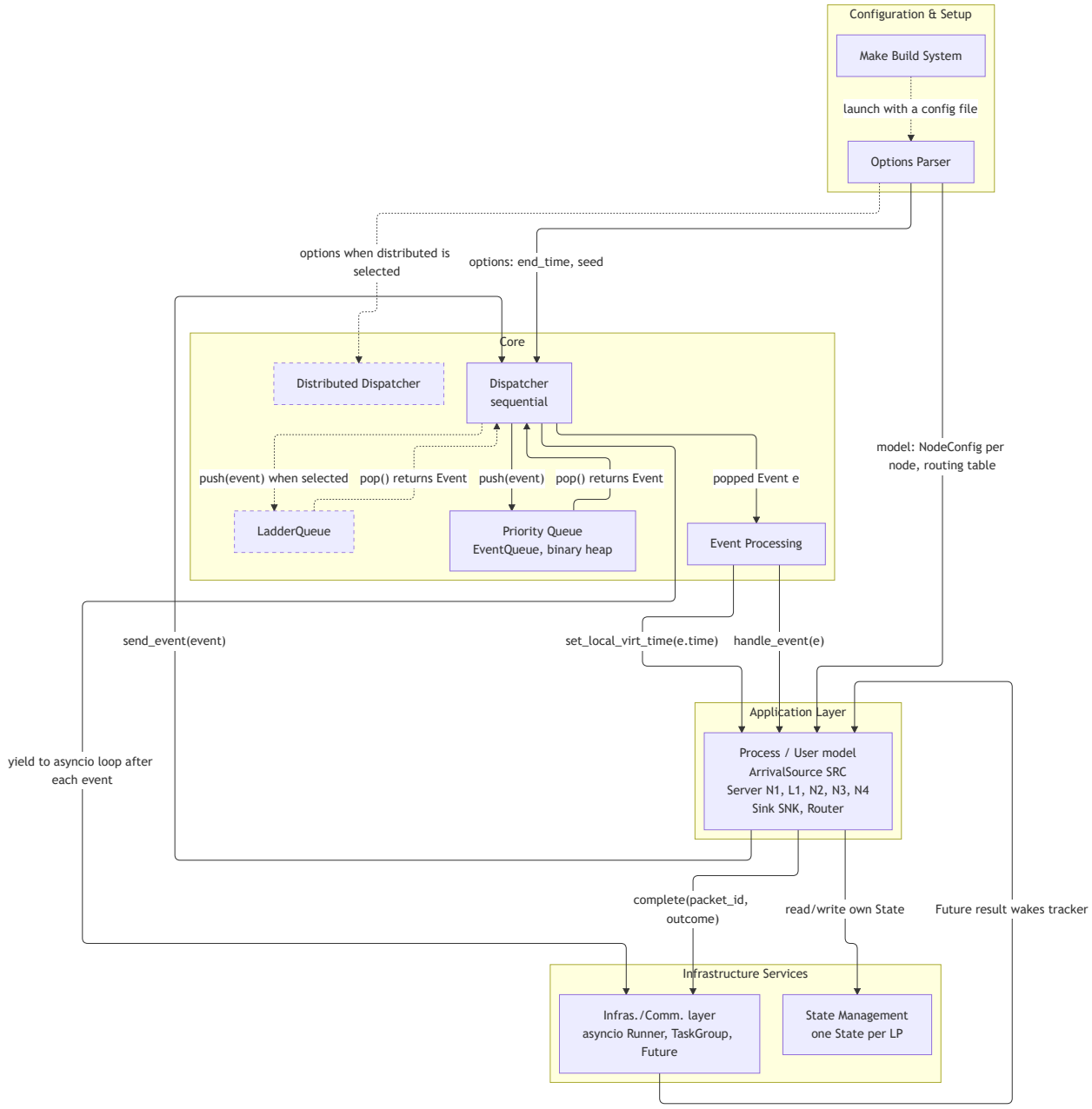


Figure X.1: AQNet component architecture.

Boundaries. Inside the simulator are the blocks of Figure X.1. Outside it are the user or experiment runner that starts a run, the analytical reference table B4 (Design 2) and the experiment matrix D3 (Design 1). The asyncio event loop is only an execution mechanism; virtual time belongs to the Dispatcher alone.

Blocks of Figure X.1:

- **Mandatory:** Make Build System and Options Parser (Configuration & Setup); Process/User model (Application Layer); Dispatcher (sequential), Event Processing and Priority Queue

(`EventQueue`) (Core); State Management and Infras./Comm. layer (Infrastructure Services).

- **Extension:** LadderQueue, an alternative priority queue with the same `push`/`pop`; and the Distributed Dispatcher, which uses per-LP queues.

Block	Responsibility	Depends on
Make Build System	Packages the simulator and provides the one command that starts a run.	–
Options Parser	Reads the run options (`end_time`, `warmup`, seed) and the model (one `NodeConfig` per node, the routing table, the source parameters); rejects invalid input before the run.	Make Build System
Process/User model	The AQNet LPs. Each LP reacts to its events through `handle_event` and reaches the rest of the simulator only through its Dispatcher reference (`send_event`, `create_completion`, `complete`, `spawn`).	Options Parser, Dispatcher, State Management, Infras./Comm. layer
Dispatcher	Pops events in key order, checks `end_time`, discards cancelled events, resets statistics at `warmup`, advances virtual time and counts `events_processed`; front end of the Future registry.	Options Parser, Priority Queue (or LadderQueue), Event Processing, Infras./Comm. layer
Priority Queue	`EventQueue`: binary heap ordered by (time, priority, seq).	–
Event Processing	Per-event step: sets the destination LP's local virtual time, then calls `handle_event`, or resolves O5 when the destination server is down (LC-07).	Process/User model, State Management
State Management	One `State` per LP. An LP writes only its own `State`; the Dispatcher only reads `up`.	–
Infras./Comm. layer	asyncio runtime (`asyncio.Runner`, one `TaskGroup`) and the Future registry keyed by `packet_id`; `Dispatcher.create_completion` and `Dispatcher.complete` are its interface.	–
LadderQueue (extension)	Alternative priority queue with the same `push`, `pop` and `__len__`.	–
Distributed Dispatcher (extension)	Alternative dispatcher with one event queue per LP.	Options Parser, Event Processing, Infras./Comm. layer

In Figure X.1 the arrow `complete(packet_id, outcome)` stands for `Dispatcher.complete`, which resolves the Future held by the Infras./Comm. layer.

X.2 Class Design

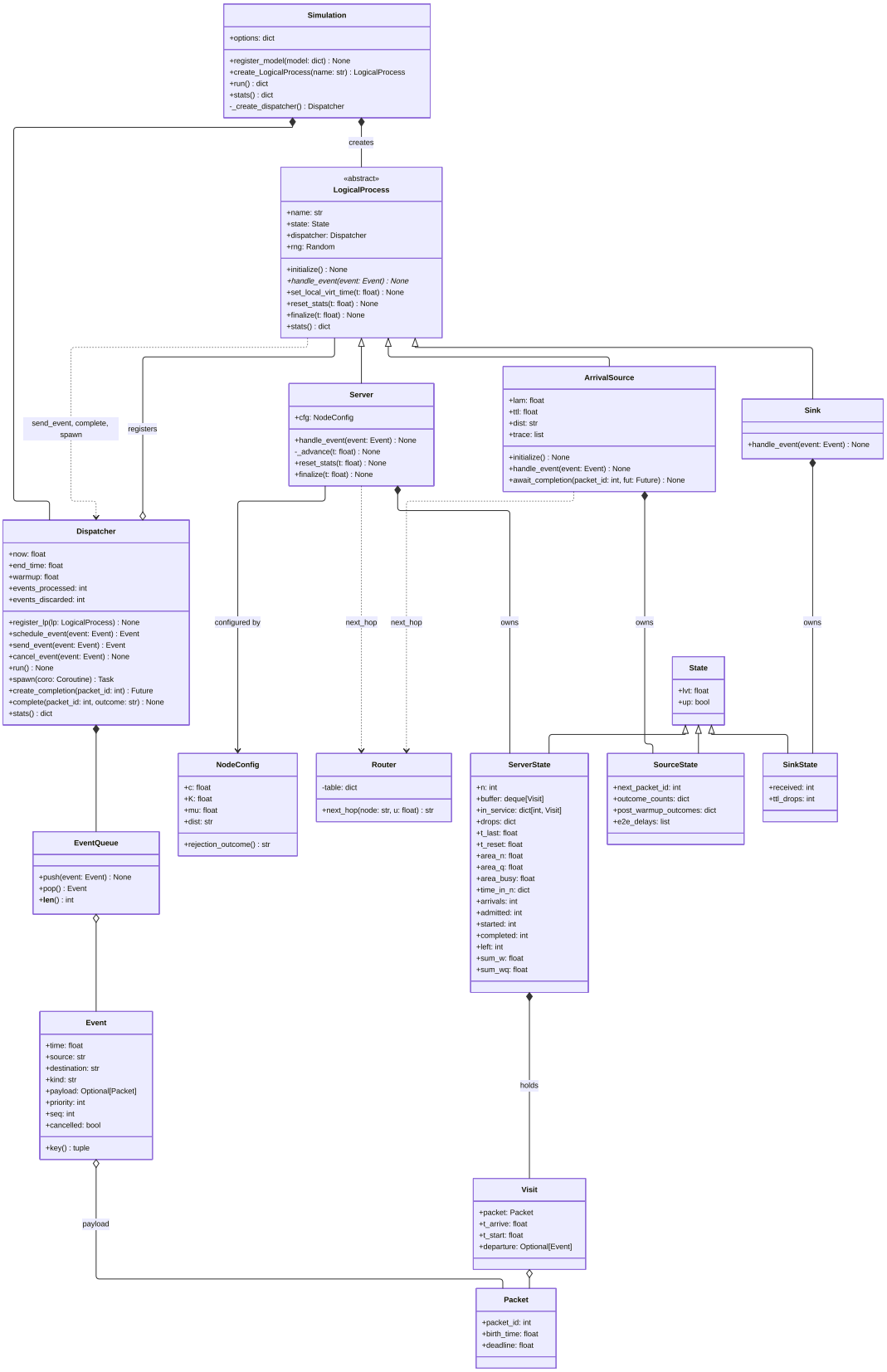


Figure X.2: AQNet class diagram.

Class	Method	Parameters (type hints)	Return	Precondition
`Event`	constructor	`time: float, source: str, destination: str, kind: str, payload: Optional[Packet], priority: int, seq: int = -1`	`Event`	$0 \leq$ `time` $< \infty$; `kind` is GENERATE, ARRIVAL, DEPARTURE or FAULT
	`key`	–	`tuple[float, int, int]`	`seq` ≥ 0 (assigned)
	field `cancelled`	`cancelled: bool = False`	–	Set only by `Dispatcher.cancel_event`
`Packet`	constructor	`packet_id: int, birth_time: float, deadline: float`	`Packet`	`deadline` $\geq$ `birth_time`
`Visit`	constructor	`packet: Packet, t_arrive: float`; fields `t_start: float = nan`, `departure: Optional[Event] = None`	`Visit`	`t_arrive` = `now`; written only by the `Server` that holds it
`EventQueue`	constructor	–	`EventQueue`	None
	`push`	`event: Event`	`None`	`event.seq` ≥ 0
	`pop`	–	`Event`	`len(queue)` > 0
	`__len__`	–	`int`	None; the dispatcher loops while it is > 0
`Dispatcher`	constructor	`queue: EventQueue, end_time: float, warmup: float, seed: int`	`Dispatcher`	$0 \leq$ `warmup` $<$ `end_time`; called by `Simulation._create_dispatcher`
	`register_lp`	`lp: LogicalProcess`	`None`	Not running; `lp.name` not yet registered. Sets `lp.dispatcher = self` and `lp.rng = random.Random(f"{seed}/{lp.name}")`, then calls `lp.initialize()`
	`schedule_event`	`event: Event`	`Event`	`now` $\leq$ `event.time`; destination registered; assigns `seq`
	`send_event`	`event: Event`	`Event`	Called inside the source LP's `handle_event`; as `schedule_event`
	`cancel_event`	`event: Event`	`None`	Event scheduled and not yet popped; sets `event.cancelled = True`; the event is discarded when popped
	`run` (async)	–	`None`	All LPs registered; called once; algorithm in §X.4.2
	`spawn`	`coro: Coroutine`	`asyncio.Task`	`run` is active; creates the task in the run's `TaskGroup`
	`create_completion`	`packet_id: int`	`asyncio.Future`	Event loop running; no pending Future for `packet_id`
	`complete`	`packet_id: int, outcome: str`	`None`	Future of `packet_id` pending; `outcome` in O1–O5; one outcome per packet
	`stats`	–	`dict`	None; includes `now`, `events_processed`, `events_discarded`
`Simulation`	constructor	`options: dict` (`end_time: float`, `warmup: float = 0`, `seed: int`)	`Simulation`	$0 <$ `end_time` $< \infty$; $0 \leq$ `warmup` $<$ `end_time`
	`register_model`	`model: dict` (one `NodeConfig` per node, the `Router`, the source parameters)	`None`	Called once, before any `create_LogicalProcess`
	`create_LogicalProcess`	`name: str`	`LogicalProcess`	Model registered; `name` in the model and not yet created
	`run`	–	`dict`	All LPs created; not yet run

Class	Method	Parameters (type hints)	Return	Precondition
	`_create_dispatcher`	–	`Dispatcher`	Called once, inside `run`
	`stats`	–	`dict`	`run` has returned
`LogicalProcess`	fields	`name: str, state: State, dispatcher: Dispatcher, rng: random.Random`	–	`dispatcher` and `rng` set by `register_lp`
	`initialize`	–	`None`	Called once, by `register_lp`, at `now` $= 0$
	`handle_event` (synchronous)	`event: Event`	`None`	`event.destination == name`; `state.lvt == event.time`; called only by the dispatcher
	`set_local_virt_time`	`t: float`	`None`	$t \geq$ `state.lvt`
	`reset_stats`	`t: float`	`None`	Called once by the dispatcher with $t =$ `warmup`; `state.lvt` $\leq t$. Default does nothing; `Server` overrides
	`finalize`	`t: float`	`None`	Called once by the dispatcher with $t =$ `end_time` when the run stops. Default does nothing; `Server` overrides
	`stats`	–	`dict`	`finalize` has run
`State`	fields	`lvt: float, up: bool`	–	Written only by its own LP; the dispatcher reads `up`
`SourceState`	fields	`next_packet_id: int, outcome_counts: dict[str, int], post_warmup_outcomes: dict[str, int], e2e_delays: list[float]`	–	Owned by one `ArrivalSource`; `outcome_counts` counts every packet and is never reset; the other two count only packets with $t_b \geq t_w$
`ServerState`	fields	`n: int, buffer: deque[Visit], in_service: dict[int, Visit], drops: dict[str, int], t_last: float, t_reset: float, area_n: float, area_q: float, area_busy: float, time_in_n: dict[int, float], arrivals: int, admitted: int, started: int, completed: int, left: int, sum_w: float, sum_wq: float`	–	Owned by one `Server`
`SinkState`	fields	`received: int, ttl_drops: int`	–	Owned by one `Sink`; never reset
`ArrivalSource`	constructor	`name: str, lam: float, ttl: float, router: Router, dist: str = "exp", trace: list[tuple[float, float]] = []`	`ArrivalSource`	`dist` is `"exp"`, or `"det"` or `"trace"` in tests only; $0 < \lambda < \infty$ unless `"trace"`; a trace is a non-empty list of (birth time, τ) sorted by time; $\tau \geq 0$; `name` has a routing row
	`initialize`	–	`None`	Schedules the first GENERATE at `now` $+A$, or at the first trace time
	`handle_event`	`event: Event`	`None`	`event.kind` is GENERATE
	`await_completion` (async)	`packet_id: int, fut: asyncio.Future`	`None`	`fut` came from `create_completion(packet_id)`; started with `dispatcher.spawn`

Class	Method	Parameters (type hints)	Return	Precondition
`Server`	constructor	`name: str, cfg: NodeConfig, router: Router`	`Server`	`name` has a routing row
	`handle_event`	`event: Event`	`None`	`event.kind` is ARRIVAL, DEPARTURE or FAULT; first statement is `_advance(event.time)`
	`_advance`	`t: float`	`None`	$t \geq$ `state.t_last`; updates the areas
	`reset_stats`	`t: float`	`None`	`_advance(t)`, then every counter set to 0 and `t_reset = t`
	`finalize`	`t: float`	`None`	`_advance(t)`
`Sink`	constructor	`name: str`	`Sink`	`name` has no routing row
	`handle_event`	`event: Event`	`None`	`event.kind` is ARRIVAL
`Router`	constructor	`table: dict[str, dict[str, float]]`	`Router`	Every $p_{ij} \in [0, 1]$; each row, including the exit to SNK, sums to 1; read-only after construction
	`next_hop`	`node: str, u: float`	`str`	`node` has a row; $0 \leq u < 1$, drawn by the calling LP from its own `rng`
`NodeConfig`	constructor	`c: float, K: float, mu: float, dist: str = "exp"`	`NodeConfig`	c integer ≥ 1 or ∞; $K \geq c$; $K = \infty$ if $c = \infty$; $0 < \mu < \infty$; `dist` is `"exp"`, or `"det"` in tests only
	`rejection_outcome`	–	`str`	$K < \infty$

X.3 Sequence Diagrams

Figures X.3 and X.4 are the initialization and the event loop of the sequential dispatcher. The event loop follows the algorithm of §X.4.2: it discards cancelled events, resets statistics at `warmup`, resolves O5 without calling `handle_event` when the destination server is down (LC-07), wakes trackers through `complete` and the zero-length yield, and closes the statistics with `finalize`.

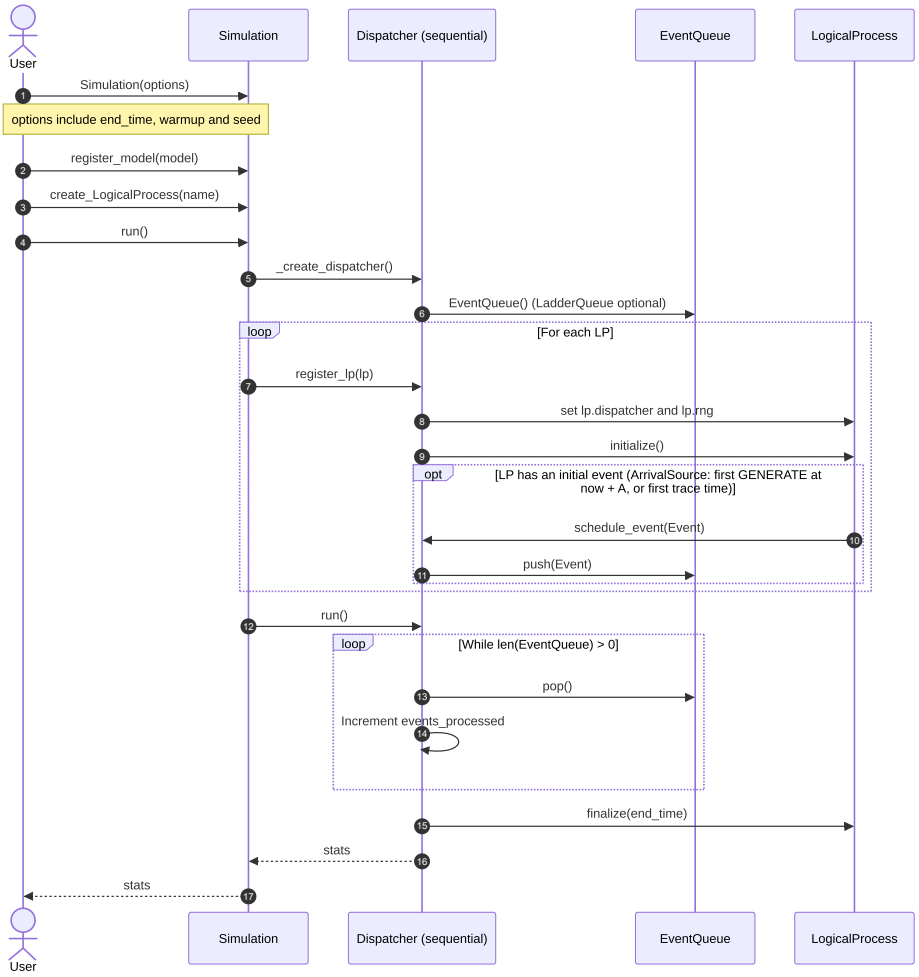


Figure X.3: Initialization.

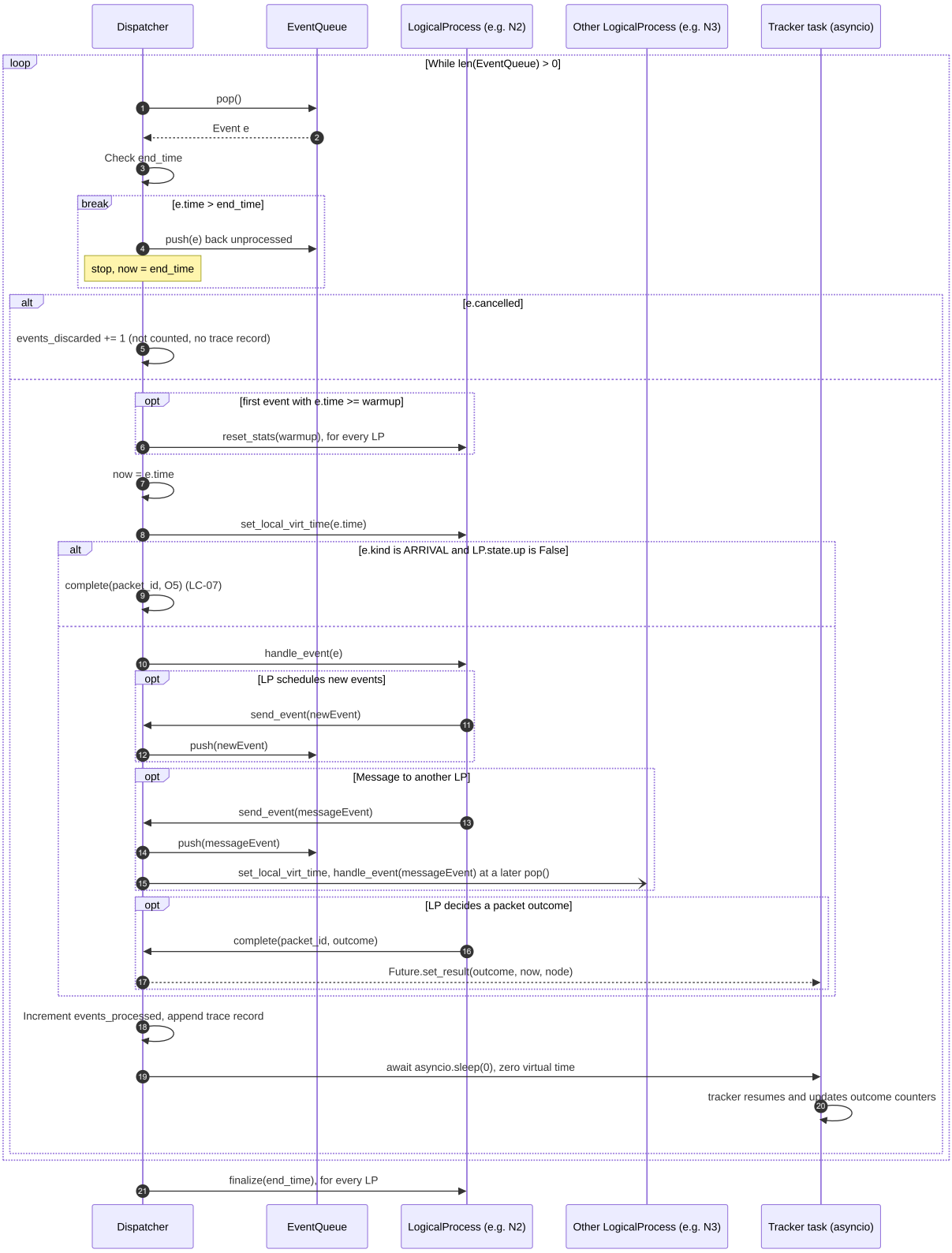


Figure X.4: Event loop.